

In The Name of God



Navid Zare (40435071)

Neural Networks and Deep Learning
Assignment 3: cGAN on Fashion-MNIST - Grayscale Image Colorization using
CAE - AAE for Latent Space Learning

1. Introduction

This report presents the design, implementation, and evaluation of three neural-network systems built entirely from scratch in NumPy, without any high-level deep-learning framework such as PyTorch or TensorFlow. The three tasks are a Conditional Generative Adversarial Network (cGAN) for generating Fashion-MNIST images, a Convolutional Autoencoder for grayscale-to-RGB image colorization, and an Adversarial Autoencoder (AAE) that adds an adversarial constraint to the colorization autoencoder's latent space.

The purpose of building everything by hand is to understand the mechanics that a framework normally hides. Every component — forward propagation, manual backpropagation, batch normalization, convolution through the `im2col/col2im` transformation, and the Adam optimizer — is written at the level of individual array operations and was checked with numerical gradient verification. The report is organized by task. For each one, the architecture and training procedure are described first, and then every plot and figure it produces is examined in detail: what the figure is meant to show, what we expected to see before running the experiment, what actually appeared, what the reported numbers mean, and what conclusion follows.

The reconstruction tasks are scored with three quantities:

1. Mean Squared Error (MSE) is the average of the squared differences between the predicted and the true pixel values; it is always positive, and smaller is better.
2. Peak Signal-to-Noise Ratio (PSNR), measured in decibels, restates that error on a logarithmic scale relative to the largest possible pixel value; larger is better, with values near 20 dB indicating a recognizable but imperfect image and values above 30 dB indicating high fidelity.
3. Structural Similarity (SSIM) ranges from 0 to 1 and compares local brightness, contrast, and structure rather than raw pixel differences; a value of 1 means the two images are identical, and values around 0.5–0.6 indicate a moderate structural match.

2. Conditional GAN on Fashion-MNIST

2.1 Architecture

A Conditional GAN trains two networks against each other. The Generator (G) turns random noise, together with a chosen class label, into an image, and the Discriminator (D) looks at an image and its label and judges whether the image is real or generated. Both networks here are fully connected (MLP-based), which is sufficient for the $28 \times 28 = 784$ -dimensional Fashion-MNIST images and keeps the from-scratch implementation manageable.

The Generator takes a 100-dimensional noise vector drawn from a standard normal distribution and concatenates it with a 10-dimensional one-hot class vector, giving a 110-dimensional input. This passes through three Dense layers of 256, 512, and 1024 units, each followed by Batch Normalization and a LeakyReLU(0.2) activation, and ends in a Dense(784) layer with a *tanh* activation that maps outputs into the $-1,1$ range. The Generator has 1,492,496 trainable parameters. The Discriminator takes the 784-pixel image concatenated with the same 10-dimensional one-hot vector (794 inputs), passes through two Dense layers of 512 and 256 units with LeakyReLU(0.2) and Dropout(0.3), and ends in a single Dense(1) unit with a Sigmoid activation that outputs the probability the image is real. The Discriminator has 538,625 trainable parameters.

Table 1. Conditional GAN architecture summary.

Component	Architecture	Parameters
Generator	Dense(110→256→512→1024→784) + BN + LeakyReLU + tanh	1,492,496
Discriminator	Dense(794→512→256→1) + LeakyReLU + Dropout(0.3) + Sigmoid	538,625
Conditioning	One-hot class vector (10-dim) concatenated in both G and D	0

Several choices in this design are deliberate. LeakyReLU is used instead of plain ReLU so that a small gradient still flows when a unit is negative, which keeps neurons from dying during the unstable early phase of adversarial training. Batch Normalization in the Generator stabilizes the scale of the activations from layer to layer. Dropout in the Discriminator weakens it slightly on purpose, so that it does not crush the Generator and leave it without a useful learning signal. The class label is supplied as a simple one-hot vector that carries no trainable parameters of its own, and the tanh output is chosen to match the $-1,1$ range into which the pixels were normalized.

2.2 Training Methodology

Each training step has two stages. First the Discriminator is updated: it sees a batch of real images with the target label 1.0 and a batch of generated images with the target label 0.0, the two binary cross-entropy gradients are added together, and one Adam update is applied. Second the Generator is updated: it produces a batch of fake images, these are passed

through the Discriminator with the target label 1.0, and the error is propagated back through the (frozen) Discriminator and into the Generator. Training used 15,000 images (25% of the Fashion-MNIST training set) for 15 epochs with a batch size of 128, optimized by Adam with a learning rate of 0.0002 and momentum terms $\beta_1 = 0.5$ and $\beta_2 = 0.999$. The lower β_1 makes the optimizer react more quickly to recent gradients, which suits the constantly shifting GAN objective. The sampled subset was close to evenly balanced across the ten classes, with between 1,476 and 1,546 images each.

2.3 Results and Analysis

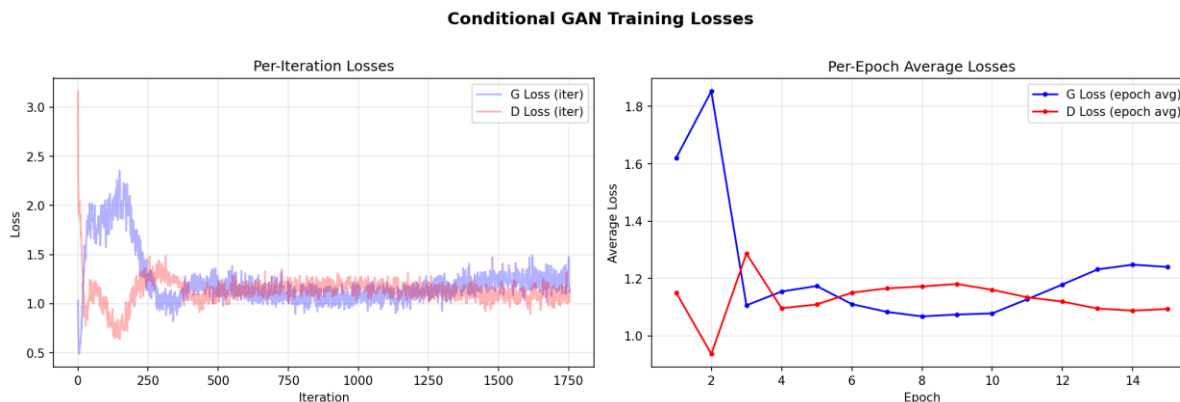


Figure 1. Generator and Discriminator training losses, shown per iteration (left) and per epoch (right).

Figure 1 records how the two competing networks performed throughout training. The left panel plots the loss of the Generator and of the Discriminator at every individual training step, which produces a noisy, jagged trace, while the right panel averages those values within each epoch so the underlying trend is easier to follow. These curves are the main tool for judging a GAN, because a GAN has no single accuracy score to report; the two networks are in a contest, and what matters is the balance between them rather than the value of either loss on its own.

Before training, the outcome we hoped for was a balance rather than a clear winner. If the Discriminator grew too strong, its loss would fall toward zero, the Generator would stop receiving useful feedback, and learning would stall. If the Generator instead overpowered the Discriminator, the Discriminator loss would climb without limit and the images would fall apart. A healthy run settles into a steady region where both losses flatten out and neither runs away. For the summed Discriminator loss used here, the point of perfect balance — where the Discriminator can no longer tell real from fake and is reduced to guessing — sits at $2 \times \ln(2) \approx 1.386$.

The curves follow that expected pattern closely. The first two epochs are turbulent: the Generator loss climbs to about 1.85 at epoch 2 while the Discriminator briefly takes the lead, its loss dipping to 0.94, and then at epoch 3 the situation reverses sharply as the Generator improves and its loss drops to about 1.11 while the Discriminator rises to 1.29. After this early skirmish the training calms, and from epoch 4 onward both losses stay within a narrow band, the Discriminator holding between roughly 1.09 and 1.18. By the

final epoch the Generator loss is 1.2400 and the Discriminator loss is 1.0938. One detail worth noticing is the steady climb of the Generator loss from its lowest point of about 1.07 near epoch 8 up to 1.24 by epoch 15, ending just above the Discriminator, which settles at 1.09. This is not a failure but the natural result of the Discriminator gradually sharpening, which forces the Generator to work a little harder to keep fooling it. The per-iteration panel shows the same story from another angle: the band of points is wide and erratic at the start and tightens into a stable horizontal stripe as training matures.

Measured against expectations, this is a clear success. The Discriminator loss of 1.09 sits just below the 1.386 balance point, which means the Discriminator keeps only a slim edge instead of dominating, and at no stage does either loss collapse to zero or explode. This is precisely the regime in which a GAN produces meaningful images, and it gives us confidence that the samples in the next two figures reflect genuine learning rather than a broken or collapsed model.

Training Progression: Generated Samples Over Epochs

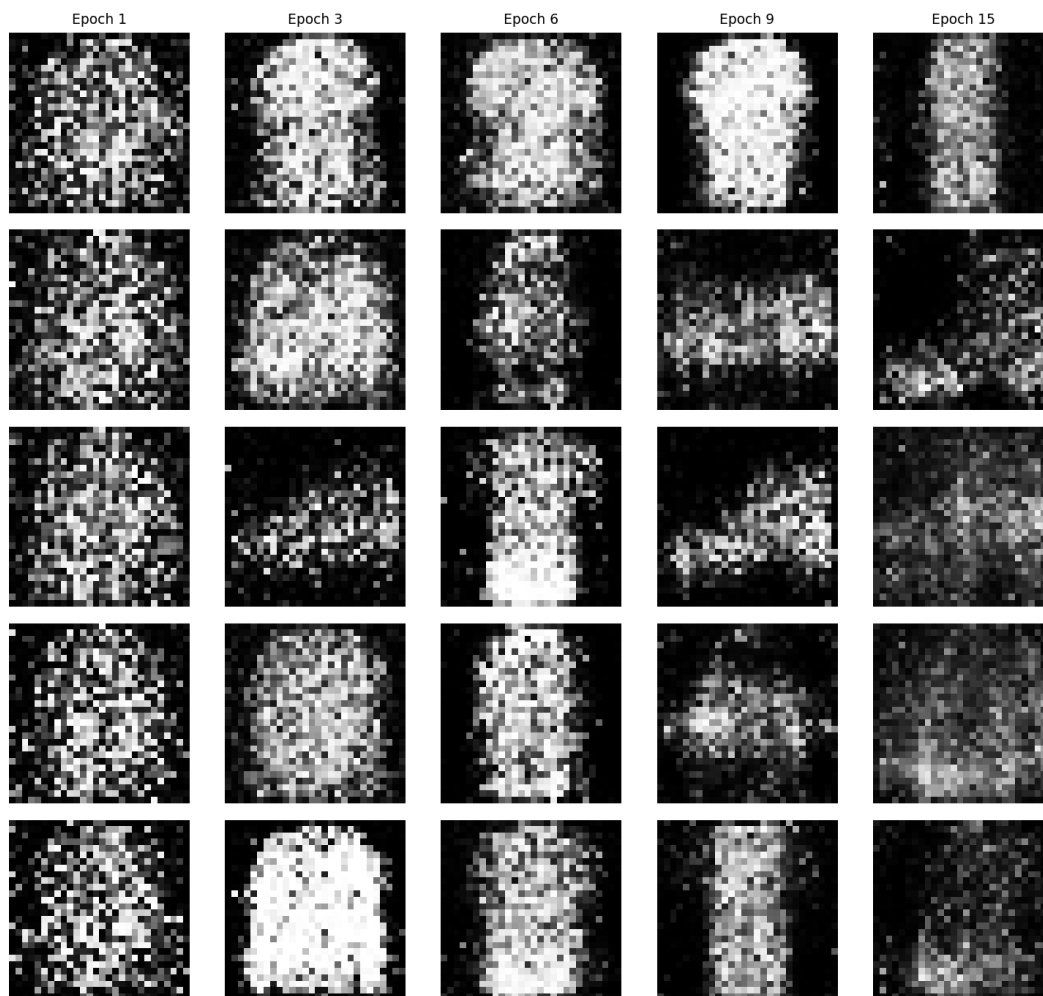


Figure 2. Generated samples for five representative classes at five points across training.

Figure 2 turns the abstract loss curves into something we can see directly. Each column is a snapshot taken at a different stage of training, moving from the earliest epoch on the left to the final epoch on the right, and each row follows a single class (T-shirt, Trouser, Sandal, Sneaker, and Bag) so the same category can be tracked as it develops. The aim is to confirm that the improvement implied by the loss curves corresponds to images that actually become more recognizable over time.

We expected the early columns to look like little more than noise or vague blobs, and the later columns to show progressively cleaner, more identifiable garment shapes. That is what appears. In the first column the outputs are faint and shapeless; by the middle columns a rough silhouette of the correct object has emerged; and in the final column each row shows a shape that can be read as its intended class. The largest visible jump happens early — the step from noise to a recognizable outline — after which the later epochs mostly refine and steady the shape rather than transform it. It is also noticeable that the broad, solid items such as the Bag and the T-shirt settle into a clear form sooner than the thin or detailed items such as the Sneaker and the Sandal, which stay noisier for longer.

The figure meets expectations and reinforces the conclusion drawn from Figure 1: the Generator is genuinely learning class-specific structure, and it is doing so gradually and stably rather than memorizing or collapsing. Seeing the shapes sharpen column by column is the most direct evidence that the training process is working as intended.



Figure 3. Final generated samples at epoch 15. Each row corresponds to one of the ten Fashion-MNIST classes.

Figure 3 is the main deliverable of the first task: a full sheet of images generated at the end of training, with each of the ten rows devoted to one class and each column an independent sample of that class. Two things matter when reading it. First, does each row actually contain the garment it is supposed to, in other words is the class conditioning working? Second, do the samples within a row vary, or has the Generator collapsed to producing the same picture over and over?

Our expectation was that the conditioning would hold, so every row would show its own category, and that there would be visible variety across each row, since mode collapse is the most common way a GAN fails. The figure confirms both. Every row reproduces the correct type of object: the upper-body garments (T-shirt, Pullover, Coat, Shirt) appear as filled vertical silhouettes, trousers and dresses as narrower elongated shapes, and the footwear classes as the expected sideways forms. Within each row the individual samples differ in pose and detail, so there is no sign of collapse. The images are visibly grainy,

which is the expected price of using fully connected layers rather than convolutions, but they are clearly the right objects.

The per-class pixel statistics in Table 2 put numbers to what the eye sees. The mean-pixel column is the average brightness of the generated images for that class on the normalized 0,1 scale, and the std-pixel column measures how much the brightness varies within the image; together they describe how much of the frame the object fills and how strongly it stands out from the black background.

Table 2. Per-class statistics of the generated samples at epoch 15.

Class	Mean Pixel	Std Pixel
T-shirt/top	0.2796	0.3209
Trouser	0.2796	0.3017
Pullover	0.2938	0.3607
Dress	0.2762	0.3036
Coat	0.2685	0.3193
Sandal	0.3100	0.2992
Shirt	0.2845	0.3080
Sneaker	0.2599	0.2746
Bag	0.3485	0.3405
Ankle boot	0.3348	0.3588

Reading down the table, the classes with the highest mean brightness are Bag (0.3485), Ankle boot (0.3348), Sandal (0.3100), and Pullover (0.2938), while the lowest belong to Sneaker (0.2599), Coat (0.2685), Dress (0.2762), and Trouser (0.2796). The values are clustered fairly tightly, most of them between about 0.26 and 0.35, which indicates that the Generator fills the frame to a broadly similar degree across categories rather than strongly favoring a few classes. The most solid shapes, Bag and Ankle boot, remain the most strongly filled, while the sparser Sneaker is the least, in keeping with the thinner shapes appearing a little noisier in the image grid. The overall message of the figure is positive: the central goal of the task, generating a specified class on demand, is achieved for all ten categories, with only the expected loss of fine detail.

2.4 Discussion

Taken together, the three cGAN figures tell a consistent story. The loss curves show a stable contest that settles near the theoretical balance point without collapse, the progression grid shows shapes sharpening steadily over the epochs, and the final sheet shows correct, varied samples for every class. The performance matches what one should expect from a fully connected conditional GAN trained on a quarter of the data for fifteen epochs: reliable class control and clear silhouettes, but limited fine texture.

Discussion questions. A standard GAN maps noise to images with no control over the category, whereas a conditional GAN feeds a class label into both networks and can therefore generate a chosen class on command; here the label is a 10-dimensional one-hot vector concatenated with the Generator's noise and with the Discriminator's image. The classes the Generator filled most confidently were the broad, solid garments such as Bag, Ankle boot, and Pullover, and the sparser or thinner items such as Sneaker, Dress, and Trouser were reproduced a little less crisply. Over training the samples moved from noise at epoch 1 to vague silhouettes by the middle epochs to recognizable, class-specific shapes by epoch 15, and the Generator and Discriminator losses settled into a balanced equilibrium near 1.24 and 1.09 with no collapse. The most effective improvements would be the ones listed below.

The natural next steps would be a convolutional (DCGAN) Generator and Discriminator for sharper spatial detail, training on the full 60,000-image set for more epochs, a more stable objective such as a Wasserstein loss with gradient penalty, and one-sided label smoothing on the real targets to keep the Discriminator from becoming over-confident.

3. Convolutional Autoencoder for Image Colorization

3.1 Preprocessing

The colorization dataset contains 490 training images and 10 test images, all natural RGB photographs. Each image was resized to 64×64 pixels and scaled into the 0,1 range. Grayscale versions were produced from the color originals with the standard luminance formula $Y = 0.2989 \cdot R + 0.5870 \cdot G + 0.1140 \cdot B$. The grayscale image is the input and the original color image is the target the network must reproduce, which makes this a self-supervised task: no human labelling is needed, because the color answer already exists in the data.

3.2 Architecture

The encoder has three blocks, each a 3×3 convolution with padding 1 (which keeps the spatial size unchanged), followed by Batch Normalization, a ReLU activation, and a 2×2 max-pooling step that halves the height and width. The number of channels grows $1 \rightarrow 16 \rightarrow 32 \rightarrow 64$ while the resolution shrinks from 64×64 down to 8×8, compressing the image into a compact summary. The decoder reverses this: three blocks of nearest-neighbor upsampling followed by a 3×3 convolution, with Batch Normalization and ReLU in the first two blocks and a Sigmoid activation in the last to force the output back into [0, 1]. The channels shrink 0,1 while the resolution returns to 64×64. The model has 47,139 trainable parameters, which is small by modern standards.

Table 3. Convolutional Autoencoder architecture and output shapes.

Stage	Layer sequence	Output shape
Input	Grayscale image	$1 \times 64 \times 64$
Encoder block 1	Conv(1→16) + BN + ReLU + MaxPool(2)	$16 \times 32 \times 32$
Encoder block 2	Conv(16→32) + BN + ReLU + MaxPool(2)	$32 \times 16 \times 16$
Encoder block 3	Conv(32→64) + BN + ReLU + MaxPool(2)	$64 \times 8 \times 8$
Decoder block 1	Upsample(2) + Conv(64→32) + BN + ReLU	$32 \times 16 \times 16$
Decoder block 2	Upsample(2) + Conv(32→16) + BN + ReLU	$16 \times 32 \times 32$
Decoder block 3	Upsample(2) + Conv(16→3) + Sigmoid	$3 \times 64 \times 64$

3.3 Training Methodology

The model was trained for 10 epochs with a batch size of 8, using Adam with a learning rate of 0.001. The objective is the pixelwise Mean Squared Error between the predicted and target color images, $L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$, where N counts every pixel value across all three channels and all images. The ten test images were used to track validation loss during training.

3.4 Results and Analysis

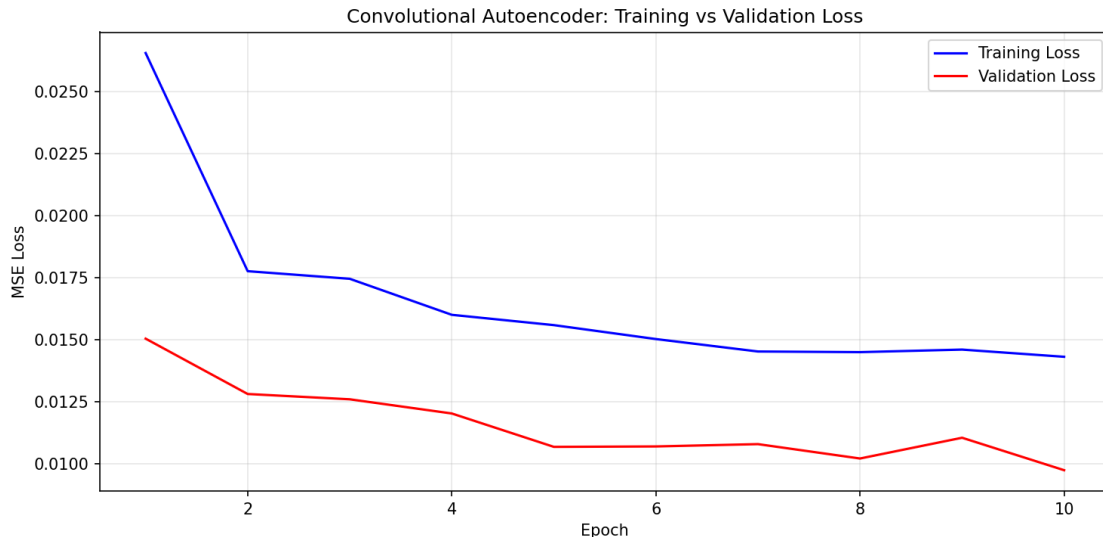


Figure 4. Training and validation MSE loss over 10 epochs.

Figure 4 plots the average error on the training images and on the held-out test images at the end of each epoch. The vertical axis is the Mean Squared Error, so lower is better, and the horizontal axis is the epoch number. The plot confirms two things at once: that the network is actually learning (the curves should fall), and that it is not simply memorizing the training set at the expense of new images (the two curves should not diverge, with the validation curve climbing while the training curve keeps dropping).

We expected both curves to decrease quickly at first and then flatten as the model approached the limit of what it could learn from 490 images. The training loss behaves exactly this way, falling smoothly from 0.0265 in the first epoch to 0.0143 in the last. The validation loss also falls, from 0.0150 to 0.0097, and then levels off. There is one small bump where the validation loss rises slightly at epoch 9, to about 0.0110, before dropping to its lowest value at epoch 10, which is normal short-term noise from such a tiny test set and not a cause for concern.

The one feature that stands out is that the validation loss sits below the training loss for the whole run, which is the reverse of the usual situation. This has a straightforward explanation and is not a sign of error. The test set contains only ten images and happens to hold scenes that are somewhat easier to color than the average training image, so the score on them is naturally a little better. In addition, Batch Normalization adds a small amount of noise from each batch's statistics while training but uses stable averaged statistics at evaluation time, which slightly favors the validation measurement. The important point for judging the model is the direction of the curves, not their order: because the validation loss keeps falling and then settles rather than rising, the model is generalizing rather than overfitting. Overall, the training behaved as expected and reached a sensible stopping point.

Image Colorization Results on Test Set

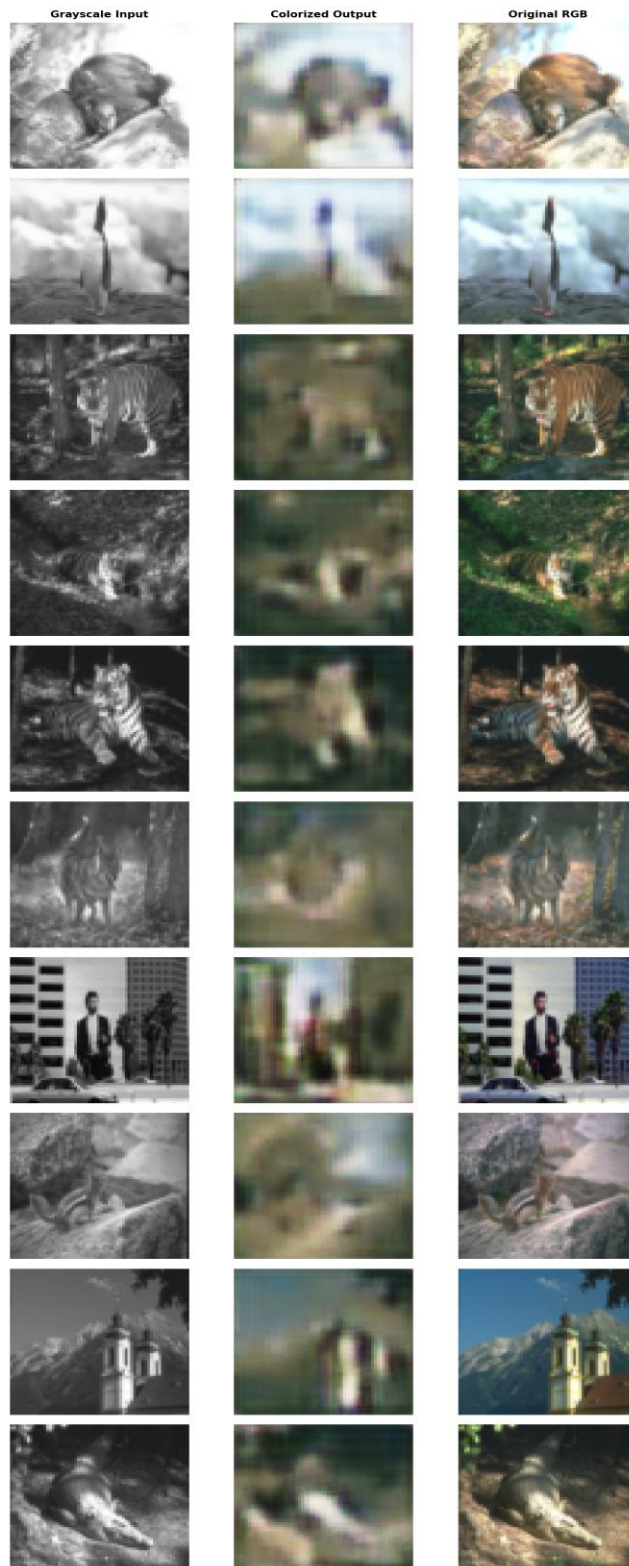


Figure 5. Colorization on the 10 test images: grayscale input (left), model prediction (centre), and original RGB (right).

Figure 5 is where the numbers become tangible. Each row shows one test image three ways: the grayscale input given to the network on the left, the network's color prediction in the middle, and the true color image on the right. Placing them side by side lets us see not just how large the error is but what kind of mistakes the model makes.

Knowing that the model is small and trained with an MSE objective, we expected it to capture the broad, predictable colors of a scene — green for vegetation, blue for sky, earthy tones for animals — while struggling with fine detail and with regions whose color cannot be guessed from brightness alone. The predictions confirm this. The model recovers the general color scheme of most images: the tigers come out in orange and brown tones, foliage is rendered in green, and the sky and built scenes take on cooler colors. At the same time the predictions are clearly blurry and somewhat muted compared with the originals, and where the model is unsure it tends to fall back on a brownish, desaturated average rather than commit to a vivid color.

This matches expectations rather than exceeding them, and the reason for the blur is explained in Section 3.5. The figure is valuable precisely because it shows the character of the output that a single error number cannot: the model has learned a believable but cautious sense of color, getting the overall idea of each scene right while smoothing away the sharp edges and the most saturated hues.

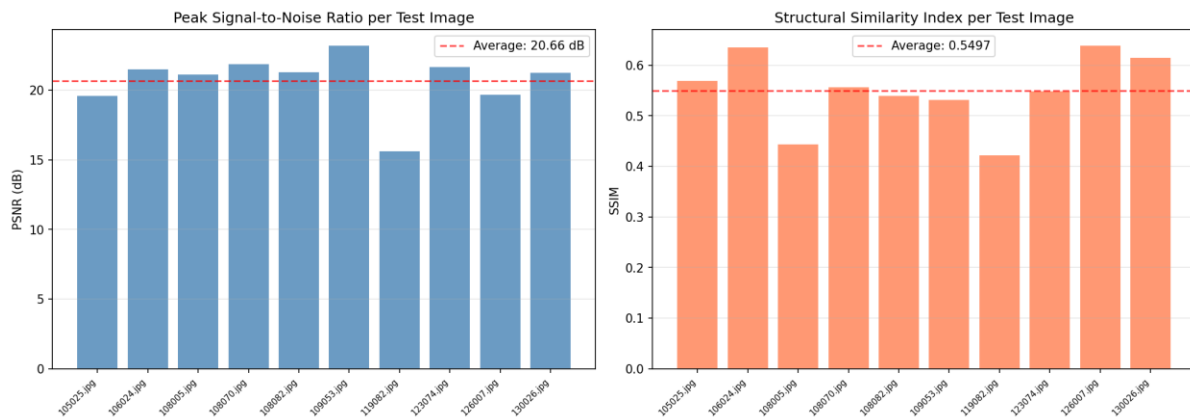


Figure 6. Per-image PSNR (left) and SSIM (right) on the test set, with the average shown as a dashed line.

Figure 6 shows the two main quality scores for each of the ten test images as bar charts, with a dashed line marking the average. The left chart is the Peak Signal-to-Noise Ratio (PSNR) in decibels, which measures raw pixel accuracy: higher bars mean the predicted pixels are closer to the true ones. The right chart is the Structural Similarity index (SSIM), which runs from 0 to 1 and measures how well the prediction preserves local patterns of brightness, contrast, and structure rather than exact pixel values. The two are shown together because they capture different ideas of quality and do not always agree, and seeing them side by side makes the per-image difficulty visible at a glance. The exact figures are listed in Table 4.

Table 4. Per-image quantitative evaluation on the test set.

Image	MSE	PSNR (dB)	SSIM
105025.jpg	0.0111	19.56	0.5688
106024.jpg	0.0071	21.47	0.6350
108005.jpg	0.0078	21.10	0.4431
108070.jpg	0.0065	21.88	0.5565
108082.jpg	0.0075	21.27	0.5395
109053.jpg	0.0048	23.18	0.5313
119082.jpg	0.0275	15.61	0.4216
123074.jpg	0.0068	21.67	0.5485
126007.jpg	0.0108	19.65	0.6381
130026.jpg	0.0075	21.25	0.6140
Average	0.0097	20.66	0.5497

Across the ten images the model averages an MSE of 0.0097, a PSNR of 20.66 dB, and an SSIM of 0.55. For a 47,000-parameter network trained on 490 images these are reasonable, middle-of-the-road scores: a PSNR around 20 dB and an SSIM around 0.55 describe an image that is clearly recognizable and roughly the right color but visibly softer than the original, which is exactly what Figure 5 showed. This is the level of performance one should expect from a model of this size, so the result meets rather than exceeds expectations.

The bars also reveal where the model struggles and where the two metrics part ways. Image 119082 is the clear outlier, scoring worst on every measure (15.61 dB and 0.4216), because its colors are complex and strongly saturated and therefore hard to infer from brightness alone. Image 109053 earns the highest PSNR (23.18 dB) yet only a middling SSIM, which happens because it contains large, smooth regions that are easy to get right pixel-by-pixel but do not carry much fine structure for SSIM to reward. Image 126007, by contrast, earns the highest SSIM (0.6381) without the highest PSNR. This split is the practical lesson of the figure: PSNR rewards getting the average color right over wide areas, while SSIM rewards preserving texture and edges, and a model can do well on one without doing well on the other.

3.5 Discussion

The dominant feature of every colorization result is a gentle blurriness, and it is worth being precise about why it appears, because it is a property of the objective rather than a bug. The MSE loss punishes the squared difference from the true color, so when the network is uncertain whether a region should be, say, green or brown, the choice that minimizes the expected squared error is not to gamble on either but to predict the average of the two. Averaging colors produces a muddy, smeared result. In other words, the blur is the mathematically optimal response to uncertainty under MSE, which is why a small model trained this way will always look soft.

Several changes would push the quality higher. A perceptual loss computed on deep image features would reward results that look right to a human rather than results that merely minimize pixel error; an additional adversarial term would encourage the network to commit to sharper, more decisive colors; a U-Net architecture with skip connections would carry fine spatial detail past the bottleneck; working in the CIE Lab color space would separate brightness from color and simplify the problem; and a larger, more varied training set would broaden what the model can recognize.

Discussion questions. The encoder–decoder structure suits colorization because the encoder distills the grayscale image into compact features that capture shape and context, and the decoder uses those features to assign color while rebuilding spatial detail. MSE was chosen for its simplicity, its smooth gradients, and its direct link to PSNR, and its weakness is the averaging-induced blur described above. The images that colorized best were those with large, uniform regions (for example 109053 and 126007), and the hardest was 119082, whose saturated and varied colors are only weakly determined by brightness. The chief limitation of a simple convolutional autoencoder is exactly this combination of MSE averaging and limited capacity, and the improvements listed above would raise the quality.

4. Adversarial Autoencoder for Latent-Space Learning (Bonus)

4.1 Architecture

The Adversarial Autoencoder keeps the idea of an encoder and a decoder but adds a third network, a discriminator that works on the latent space. The encoder compresses a color image into a 64-number latent vector and the decoder rebuilds the image from it, exactly like an ordinary autoencoder. The new discriminator is shown two kinds of latent vectors — those produced by the encoder, and random samples drawn from a chosen target distribution, here a standard Gaussian $\mathcal{N}(0, I)$ — and learns to tell them apart. By training the encoder to fool this discriminator, the model tries to make its latent codes look as if they had been drawn from the Gaussian. If this succeeds, new images can be created later simply by sampling the Gaussian and decoding.

The encoder reuses the convolutional design of Part 3 (three Conv–BN–ReLU–Pool blocks, channels $3 \rightarrow 16 \rightarrow 32 \rightarrow 64$), then flattens and passes through a Dense(4096→64) bottleneck. The decoder begins with Dense(64→4096) and ReLU, reshapes to $64 \times 8 \times 8$, and applies three upsample-and-convolution blocks mirroring the encoder. The latent discriminator is a small three-layer network: Dense(64→128) and LeakyReLU, Dense(128→64) and LeakyReLU, and Dense(64→1) with a Sigmoid. Altogether the model has 592,516 trainable parameters, of which 575,875 belong to the autoencoder and 16,641 to the discriminator.

4.2 Training Methodology

Every mini-batch is processed in three stages. In the reconstruction stage the full autoencoder is trained to minimize the MSE between input and output, updating the encoder and decoder. In the discriminator stage, samples from $\mathcal{N}(0, I)$ are labelled real and the encoder's outputs are labelled fake, and the discriminator is trained to separate them, updating only itself. In the regularization stage the encoder's outputs are passed through the discriminator with the target label real, and the error is propagated back through the (frozen) discriminator into the encoder, updating only the encoder so that it learns to produce prior-like codes. Because each backward pass needs the activations cached by its own forward pass, the encoder is re-run at the start of each stage. Three separate Adam optimizers keep independent momentum for the three updates, and the regularization update on the encoder uses a smaller learning rate of 0.0005 so that it does not overwhelm reconstruction. Training ran for 8 epochs with a batch size of 8.

4.3 Results and Analysis

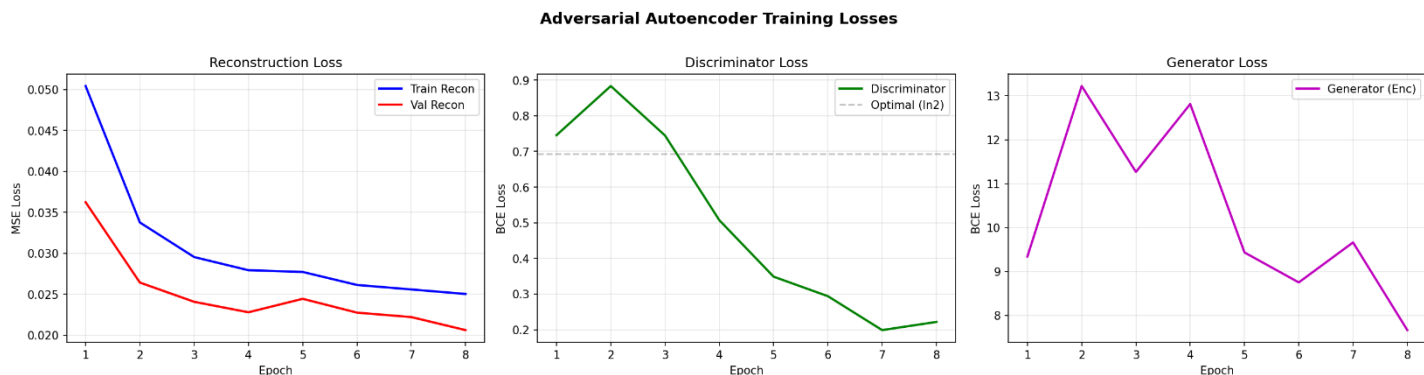


Figure 7. AAE training losses: reconstruction (left), discriminator (center), and encoder/adversarial (right) over 8 epochs.

Figure 7 separates the three objectives the AAE is juggling, one per panel. The left panel is the reconstruction loss (MSE), which measures how faithfully the decoder rebuilds the input. The center panel is the discriminator loss, which measures how well the latent discriminator tells encoded codes from Gaussian samples. The right panel is the encoder, or adversarial, loss, which measures how well the encoder is fooling that discriminator. Watching all three at once is the only way to see whether the two halves of the model — faithful reconstruction and prior-matching — are both making progress.

Our expectation, given only eight epochs, was that reconstruction would improve steadily but that the harder prior-matching goal might not be reached. The signature of successful prior-matching would be a discriminator loss settling near its balance point of $\ln(2) \approx 0.693$ (the averaged-loss equivalent of being unable to tell the two sets apart) and an encoder loss falling as the codes start to look Gaussian. The reconstruction panel behaves as hoped, with the loss dropping from 0.0504 to 0.0250. The other two panels, however, show that the adversarial game did not balance. The discriminator loss falls all the way to about 0.22, far below 0.693, which means the discriminator is easily distinguishing encoded codes from Gaussian samples — the encoder is losing the contest. Correspondingly, the encoder loss stays high and volatile, swinging between about 8 and 13 across the run and ending near 7.7. In short, the discriminator wins decisively and the encoder never learns to imitate the prior.

So this figure records a split result: the reconstruction objective met expectations while the latent-matching objective fell short, which is the outcome we anticipated for such a short training run. The value of the plot is diagnostic — it explains in advance why sampling from the prior later in Figure 11 produces poor images, because the high, flat encoder loss is direct evidence that the codes never came to resemble the Gaussian the decoder would be sampled from.

AAE Reconstruction on Test Set



Figure 8. AAE reconstruction on the 10 test images. Top row: original RGB. Bottom row: reconstruction.

Figure 8 checks the first half of the model in isolation by placing each original test image (top) directly above its reconstruction (bottom). The question here is simply how much image quality survives the trip through the 64-number bottleneck and the added adversarial pressure on the latent space.

We expected competent but slightly degraded reconstructions, a little worse than the plain autoencoder of Part 3, because the tight 64-dimensional code and the competing adversarial objective both limit how freely the encoder can preserve detail. That is what appears: the reconstructions are recognizable and keep the overall color and layout of each scene, but they are softer than the originals. The numbers in Table 5 confirm the modest drop in fidelity relative to Part 3.

Table 5. AAE test-set reconstruction metrics.

Image	MSE	PSNR (dB)	SSIM
105025.jpg	0.0274	15.62	0.5775
106024.jpg	0.0210	16.78	0.8248
108005.jpg	0.0129	18.89	0.4607
108070.jpg	0.0141	18.52	0.5040
108082.jpg	0.0177	17.53	0.6077
109053.jpg	0.0096	20.17	0.6722
119082.jpg	0.0449	13.48	0.4830
123074.jpg	0.0124	19.08	0.7143
126007.jpg	0.0242	16.16	0.5033
130026.jpg	0.0222	16.54	0.7124
Average	0.0206	17.28	0.6060

The AAE averages 17.28 dB PSNR and 0.61 SSIM, against 20.66 dB and 0.55 for the Part 3 autoencoder. The PSNR is lower, which fits the expectation that compressing into 64 numbers and adding the adversarial constraint costs some pixel accuracy. The SSIM is slightly higher than Part 3's, meaning the reconstructions preserve structure a touch better even though their pixel values are less exact; the standout is image 106024 at 0.8248. As

before, image 119082 is the most difficult (13.48 dB), the same saturated, complex scene that troubled the Part 3 model. The takeaway is that the autoencoder half of the AAE works: it reconstructs faithfully enough, with the expected trade of a little pixel accuracy for a compact, structured code.

Latent Space Analysis via PCA

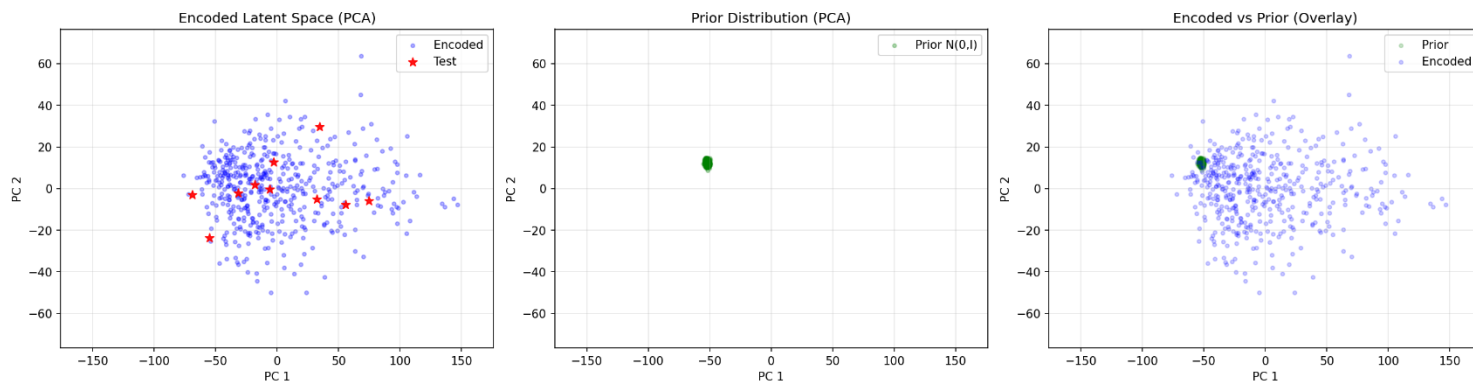


Figure 9. PCA projection of the latent space: encoded training data (left), prior $N(0,I)$ samples (center), and the two overlaid (right).

Figure 9 examines the second half of the model: did the latent codes actually come to match the target Gaussian? Because the latent space has 64 dimensions, which cannot be drawn directly, the figure uses Principal Component Analysis (PCA) to flatten it down to its two most informative directions for plotting. The left panel shows where the encoded training images land, the center panel shows where random Gaussian samples land, and the right panel overlays the two. If the regularization had worked, the two clouds would sit on top of each other; if it had not, they would occupy different regions.

The expectation set up by Figure 7 was that the two clouds would not match, and that is exactly what we see. The encoded points on the left are spread very widely, reaching out to roughly ± 100 along the principal directions, while the Gaussian samples in the center form a small, tight cluster near the origin. The overlay makes the mismatch unmistakable: the encoded distribution dwarfs the prior and spills far outside it. This confirms, in a single picture, that the encoder is placing its codes in a much larger and differently shaped region than the Gaussian the model was supposed to learn, which is the visual counterpart of the high encoder loss in Figure 7.

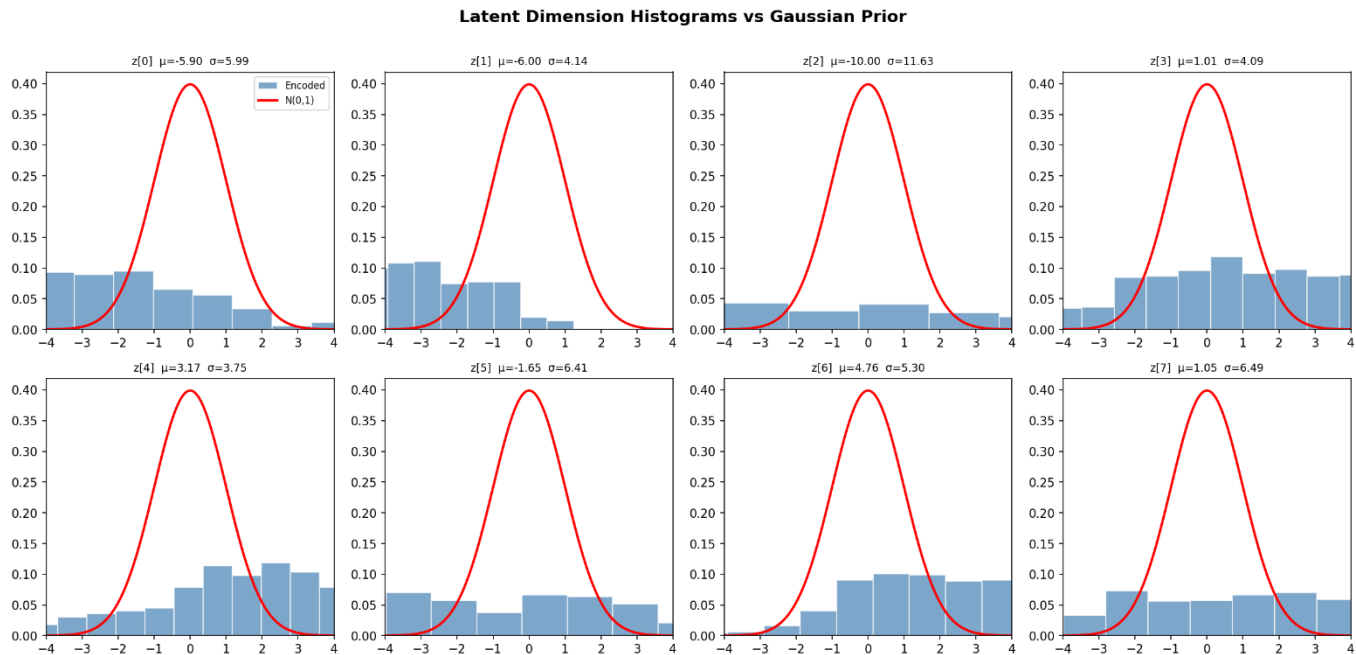


Figure 10. Per-dimension histograms of the first eight encoded latent dimensions overlaid with the standard Gaussian (red).

Figure 10 looks at the same mismatch one dimension at a time. Each of the eight panels is a histogram of the values that one latent dimension takes across the training set (the blue bars), drawn on top of the bell curve of the standard Gaussian $N(0, 1)$ those values were meant to follow (the red line). The mean (μ) and standard deviation (σ) printed above each panel summarize where that dimension is centered and how spread out it is. For a perfect match every histogram would sit under the red curve, centered at zero with a spread of one.

They do not. Some dimensions are reasonably centered but far too wide — $z[3]$, for instance, has $\mu = 1.01$ and $\sigma = 4.09$, so it is roughly in the right place but about four times too spread out. Others are pushed strongly off-center, such as $z[2]$ at $\mu = -10.00$, with an enormous $\sigma = 11.63$, and $z[1]$ at $\mu = -6.00$, sitting almost entirely to one side of the red curve. Across all 64 dimensions the average spread is 9.73, compared with the target of 1.0. The pattern is consistent with Figure 9 and pins down the problem precisely: dimension by dimension, the encoded codes are both shifted and far too broad, so they bear little resemblance to the Gaussian prior. This dimension-level view also makes clear why the next figure fails — the decoder has only ever been shown codes with values stretching to ± 10 or more, so it has no idea what to do with the small numbers a true Gaussian sample would provide.



Figure 11. Images generated by sampling latent vectors from the prior $N(0, I)$ and decoding them.

Figure 11 carries out the generative test the whole adversarial machinery was built for: draw fresh latent vectors straight from the Gaussian prior, decode them, and see whether new, plausible images appear. This is the payoff that a successful AAE is supposed to deliver.

Given everything the previous two figures showed, we expected this to fail, and it does. The decoded images are near-uniform olive and brown blobs with no recognizable structure. The reason is the mismatch documented in Figures 9 and 10: the decoder was trained on encoded codes whose values commonly reach ± 10 , but the Gaussian samples it is now handed cluster tightly around ± 1 . Those small values fall in a region of the latent space the decoder essentially never visited during training, so it has learned nothing useful to map them to and falls back on a flat, average-colored output. This is a negative result, but an informative one: it is the direct, visible consequence of the encoder failing to match the prior, and it shows clearly that prior-based generation cannot work until that matching is achieved.

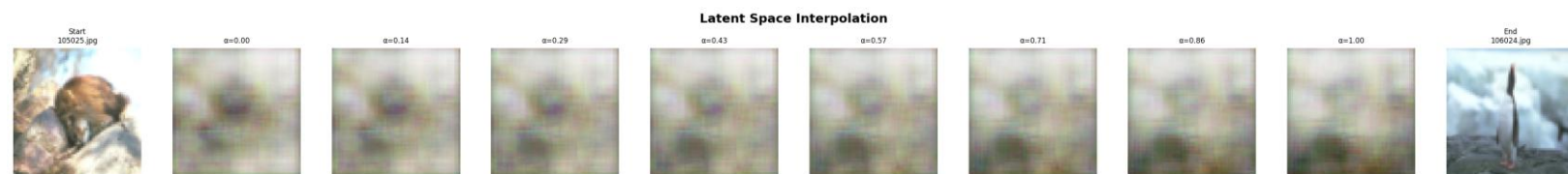


Figure 12. Latent-space interpolation between two test images, decoded at equally spaced points along the straight line between their codes.

Figure 12 probes the latent space in a gentler way. Instead of jumping to random Gaussian points, it takes two real test images, encodes both, and walks in a straight line from one code to the other, decoding the image at each step. The far-left and far-right images are the two endpoints, and the frames between them are the intermediate blends. This tests whether the latent space is smooth and meaningful in the region where the encoder actually places its codes.

Here we expected success even though prior sampling failed, and that is what happens: the sequence changes smoothly and gradually from the first image to the second, with each step a sensible blend of its neighbors rather than an abrupt jump, although the frames carry the same softness seen in the reconstructions. The contrast with Figure 11 is the important insight. Interpolation works because the path stays inside the populated region of the latent space — it travels between two points the encoder genuinely produced — whereas prior sampling failed because it ventured into the empty region near the origin. In other words, the latent space is locally well-organized and continuous even though it does not, as a whole, match the Gaussian prior. That nuance is the most interesting result of the bonus task: partial success on a hard objective, with a clear explanation for both the part that worked and the part that did not.

4.4 Discussion

The four AAE figures together describe a model that learned to reconstruct and to organize its latent space locally, but not to reshape that space into the target Gaussian within eight epochs. This is the expression of a genuine tension at the heart of the method: the reconstruction objective wants rich, high-variance codes that preserve as much information as possible, while the adversarial objective wants compact, unit-scale codes that match the prior, and reconciling the two takes careful balancing and far more training than was used here. Reconstruction and interpolation succeed; prior-matching, and therefore generation from the prior, does not.

The most promising remedies are to train much longer, on the order of 50 to 100 epochs, to ramp the weight of the adversarial loss up gradually from zero so reconstruction can establish itself first, to replace the latent discriminator with a Wasserstein critic for a smoother and more informative regularization signal, to normalize the encoder output before the discriminator, and to use a larger and more varied dataset.

5. Conclusion

This report implemented and evaluated three generative models from scratch in NumPy and examined every figure each one produced. The Conditional GAN learned to generate recognizable, correctly conditioned Fashion-MNIST images and reached a stable adversarial balance after fifteen epochs, with final Generator and Discriminator losses of about 1.24 and 1.09 and no sign of mode collapse. The Convolutional Autoencoder learned grayscale-to-RGB colorization to an average PSNR of 20.66 dB and SSIM of 0.55, recovering believable global colour while showing the soft, averaged appearance that an MSE objective necessarily produces. The Adversarial Autoencoder reconstructed images at an average PSNR of 17.28 dB and produced smooth latent interpolations, but its encoded codes did not converge to the Gaussian prior within the limited training budget — the average latent spread reached 9.73 against a target of 1.0 — so sampling from the prior yielded only featureless blobs.

Read together, the figures do more than report scores: they show why each model behaves as it does. The GAN loss curves explain the quality of the samples, the colorization comparisons explain the meaning of the PSNR and SSIM numbers, and the AAE's latent plots explain exactly why prior sampling failed while interpolation succeeded. The lasting value of the assignment is in building and verifying every gradient by hand, from the im2col convolution backward pass to the three-stage AAE training loop, and then watching the training behaviour that those gradients produce, which gives an understanding of these models that a framework would otherwise keep out of view.